

Cartographie de NOALYSS — préalable au portage Crystal

Table des matières

Résumé exécutif	2
Architecture actuelle	2
Bases de données	2
Domaines fonctionnels	3
Routage	3
Points d'entrée	4
Couche d'accès aux données	4
Dépendances externes (Composer)	5
Difficultés identifiées	5
Atouts pour le portage	6
Estimation d'effort	6
Décisions à trancher avant d'écrire du code	7
Références	7

Résumé exécutif

NOALYSS est une plate-forme de comptabilité / ERP multi-dossiers, sous GPL v2 « ou toute version ultérieure », écrite en PHP procédural-objet sur PostgreSQL.

Élément	Volume	Commentaire
PHP applicatif	~147 000 lignes (782 fichiers)	hors <code>vendor/</code> et hors tests
Tests PHPUnit	17 768 lignes (68 fichiers * <code>Test.php</code>)	oracle de portage réutilisable
JavaScript	36 942 lignes (189 fichiers)	maison, pas de framework SPA
Schéma SQL	122 tables + 97 fonctions PL/pgSQL	logique métier partiellement en base
Routes	200 entrées <code>menu_ref</code> → 75 fichiers <code>.inc.php</code>	routage piloté par la base
Langues	fr (défaut), en_US, nl_NL	gettext (.po)

Le point important : une part significative du métier comptable vit déjà en PL/pgSQL (contrôle d'équilibre, insertion en journal, lettrage, TVA, séquences, arbre de plan comptable). Cette couche est **portable telle quelle** et constitue le socle de sécurité du portage.

Architecture actuelle

Bases de données

Deux niveaux, et c'est structurant :

`account_repository`

base d'administration unique — 11 tables : `ac_dossier`,

`ac_users`, `jnt_use_dos`, `audit_connect`, `theme`, `modeledef`, `recover_pass`, `user_global_pref`, `progress`, `dossier_sent_email`, `version`.

dossier<N>

une base PostgreSQL par dossier comptable (société), créée

depuis un modèle. Deux modèles livrés :

- `include/sql/mod1` — 122 tables, dont le schéma `tva_belge` (12 tables : déclaration TVA, intra-comm, assujetti, représentant) → **modèle belge**
- `include/sql/mod2` — 110 tables, identique à `mod1` moins `tva_belge` → **modèle français**
Chaque modèle se compose de `schema.sql`, `data.sql`, `function.sql`, `constraint.sql`, `sequence.sql`, `index.sql`, `view.sql`.

Le multi-tenant est donc réalisé par **isolation physique** (une base par société), pas par une colonne de discriminant. Ce choix est à conserver ou à trancher explicitement — voir Décisions à trancher avant d'écrire du code.

Schémas PostgreSQL utilisés dans un dossier : `public`, `comptaproc` (procédures métier), `tva_belge`.

Domaines fonctionnels

Préfixe	Domaine	Fichiers
<code>acc_*</code> , <code>compta_*</code>	Comptabilité générale : journaux (achat, vente, financier, OD), opérations, lettrage	<code>acc_ledger</code> (3 729 l.), <code>acc_ledger_purchase</code> (2 291 l.), <code>acc_ledger_sale</code> (1 919 l.), <code>acc_ledger_fin</code> (1 180 l.), <code>acc_operation</code> (1 473 l.), <code>lettering</code> (926 l.)
<code>anc_*</code>	Comptabilité analytique : plans, postes, clés de répartition, balances	11 modules, <code>anc_operation</code> (1 098 l.)
<code>fiche*</code>	Fiches (tiers : clients, fournisseurs, articles, banques) avec attributs dynamiques	<code>fiche</code> (2 012 l.), <code>fiche_def</code> (941 l.)
<code>impress_*</code> , <code>balance*</code>	Éditions : balances, grand livre, bilan, âge des dettes	6 + 4 modules
<code>stock_*</code>	Stocks et dépôts	5 modules
<code>action*</code> , <code>follow_up</code>	Gestion (GED, suivi, CRM léger)	<code>follow_up</code> (2 238 l.), <code>document</code> (1 998 l.)
<code>tva_*</code> , <code>tax_*</code>	TVA, déclarations, autres taxes	+ schéma <code>tva_belge</code>
<code>forecast*</code>	Prévisions budgétaires	
<code>user_*</code> , <code>profile_*</code>	Utilisateurs, profils, droits par journal et par menu	<code>noalyss_user</code> (2 201 l.)

Routage

Le routage n'est pas dans le code : il est **en base**.

```
CREATE TABLE public.menu_ref (
    me_code text NOT NULL,      -- code de route, ex. 'ANCHOP'
    me_menu text,               -- libellé
    me_file text,               -- ex. 'anc_history.inc.php'
    me_url text,
    me_parameter text,
    me_javascript text,
    me_type varchar(2)          -- ME (menu) / PL (plugin) / PR (profil)
);
```

html/do.php lit ?ac=CODE, résout le chemin de menu via les vues v_menu_profile / v_menu_description, vérifie le droit via profile_menu, puis include le fichier .inc.php correspondant.

Répartition : 59 ME, 44 PL, 2 PR — 200 routes au total.

Conséquence pour le portage : la table de routage et les droits associés sont des **données**, pas du code. Elles se reprennent telles quelles.

Points d'entrée

do.php (12 ko)

routeur principal de l'application dossier

index.php (13 ko)

accueil / sélection de dossier

admin-noalyss.php

administration (dossiers, utilisateurs, modèles)

ajax.php + ajax_misc.php (25 ko)

98 handlers AJAX dans include/ajax/ (11 212 l.)

export.php

49 modules d'export dans include/export/ (5 619 l.) — PDF, CSV, ODS, XML

install.php (25 ko)

installation / mise à jour du schéma

login.php, user_login.php, logout.php

authentification (avec TOTP via chillerlan/php-authenticator)

compute.php, direct.php, popup.php, recherche.php, mobile.php, fid*.php, extension.raw.php

annexes

Couche d'accès aux données

include/lib/database_core.class.php (1 177 l.) est un **wrapper mince sur pg_*** : exec_sql, get_array, get_row, get_value, start/commit/rollback, gestion des *large objects* (lo_read, lo_write), séquences, introspection (exist_table, exist_column ...).

Pas d'ORM. 750 requêtes SQL sont écrites en dur dans include/class/ et include/lib/. Les requêtes sont paramétrées (\$1, \$2), ce qui limite le risque d'injection et **facilite la transposition littérale** vers Crystal.

Dépendances externes (Composer)

Paquet	Rôle
Équivalent Crystal à prévoir	horstoeko/zugferd
Facture électronique ZUGFeRD / Factur-X	à écrire (XML + PDF/A-3)
smalot/pdfparser	Lecture de PDF
prod-crystal/pdf2text ?	android/qr-code
QR codes	shard existant
chillerlan/php-authenticator	TOTP (2FA)
prod-crystal/jose ? sinon ~200 l.	phpmailer/phpmailer
Envoi de courriel	shard SMTP
php-curl-class/php-curl-class	HTTP client
HTTP::Client (stdlib)	(embarqué) include/tfpdf
Génération PDF, 3 681 l.	prod-crystal/pdf
(embarqué) html/securimage	CAPTCHA, 1 606 l.
à trancher (utile ?)	

Plusieurs de ces briques existent déjà dans prod-crystal/ (pdf, pdf-a, pdf-signature, combine-pdf, i18n, kemal-auth — renommé depuis authn —, crystal-sepa).

Difficultés identifiées

#	Difficulté	Gravité
1	Logique et présentation mélangées : 979 echo répartis dans 76 des 145 classes de include/class/. Les classes métier produisent du HTML. Il n'y a pas de couche de vue à extraire — elle est à réécrire .	Élevée
2	État global : \$g_user, \$cn, \$g_parameter, \$_SESSION utilisés directement dans 100 fichiers hors html/. Chaque unité portée doit recevoir son contexte explicitement.	Élevée
3	36 942 lignes de JavaScript maison (189 fichiers), dont function_javascript.php (2 823 l.) qui génère du JS depuis PHP . C'est un chantier à part entière.	Élevée
4	Une base par dossier : la couche d'accès doit gérer un pool de connexions par tenant, pas une connexion unique.	Moyenne

#	Difficulté	Gravité
5	228 patches SQL incrémentaux (<code>include/sql/patch/</code>) : la migration des données existantes suppose un dossier déjà à jour en DBVERSION 208.	Moyenne
6	Spécificités fiscales belges (schéma <code>tva_belge</code> , 12 tables + formulaires de déclaration) : métier réglementaire, à porter à l'identique et à tester finement.	Moyenne

Atouts pour le portage

1. **97 fonctions PL/pgSQL** portent déjà le cœur comptable (`check_balance`, `insert_jrn`, `jrn_ins`, `tva_insert`, `account_compute`, `get_pcm_tree`, `is_closed`, `find_période` ...). Elles ne sont pas à réécrire : elles sont à **conserver**. C'est le filet de sécurité du portage.
2. **68 fichiers de tests PHPUnit** (17 768 l.) couvrant `acc_ledger`, `acc_ledger_purchase`, `fiche`, etc. Ils constituent une **spécification exécutable** : chaque test porté en spec Crystal valide le portage.
3. **Routage et droits en base** : reprise sans réécriture.
4. **SQL paramétré** : transposition quasi littérale.
5. **Schéma stable et documenté** (`COMMENT` `ON` présents sur les tables/colonnes).

Estimation d'effort

Le portage intégral à iso-fonctionnalité représente de l'ordre de **150 000 lignes de PHP + 37 000 lignes de JS**. Ce n'est pas un projet de quelques semaines. Une approche par tranches verticales est la seule viable.

Découpage proposé, par ordre de valeur décroissante :

Lot	Contenu	Poids
0	Socle : connexion multi-tenant, contexte de requête, authentification, layout, i18n	~5 %
1	Plan comptable, fiches (tiers), périodes, journaux — le référentiel	~15 %
2	Saisie et consultation des opérations (achat, vente, financier, OD) + lettrage	~25 %
3	Éditions : balances, grand livre, bilan, FEC	~15 %
4	TVA et déclarations (BE + FR)	~10 %
5	Comptabilité analytique	~10 %
6	Stocks, prévisions, GED / suivi	~15 %
7	Administration multi-dossiers, import/export, extensions	~5 %

Le lot 0 + lot 1 constitue le premier jalon démontrable : se connecter à un dossier existant et consulter son plan comptable et ses fiches, **sur la base de production PostgreSQL actuelle, sans migration.**

Décisions à trancher avant d'écrire du code

NOTE Section conservée pour l'historique de l'analyse du 14 septembre 2026. **Toutes ces questions sont tranchées par les ADR**, qui font foi.

1. **Framework — tranché** : Marten ~> 0.7 (ADR-001 D1).
2. **Schéma — tranché** : schéma repensé, rupture et outil de reprise `partiduo-migrate` (ADR-001 D1, D5).
3. **Interface — tranché** : HTMX + Bulma + Opal, dans un dépôt d'interface séparé (ADR-001 D4, ADR-005).
4. **Périmètre cible — tranché** : plan de lots de l'ADR-001, socle et modules activables (ADR-006).
5. **Licence — tranché** : AGPL-3.0-or-later (ADR-001, « Neutres »).

Références

- Dépôt analysé : `prod-github/noalyss/noalyss-app/` (clone de <https://gitlab.com/noalyss/noalyss>)
- Site amont : <https://noalyss.eu> — forum : <https://forum.noalyss.eu>
- Manuel développeur : <https://manuel-fr.noalyss.eu/>